Où on en est

Summary

- Semaine 7 : break. Semaine 8 : office hours le mardi, **midterm** (modules 1 à 3) le vendredi. Pas de nouvelle matière, donc pas de notes pour ces deux semaines.
- Mardi : module 4 (fin), filtre de Wiener et prédiction linéaire ; module 5, processus de Poisson : définition, équations de Kolmogorov, loi.
- Vendredi : module 5, propriétés, bruit impulsif de Poisson (*shot noise*).
- Exercices M4 : 1, 4 (mardi) ; 5, 8, 9 (vendredi).

Key idea

Conseils du Moodle pour l'exercice M4 4.

- **4 b** : s'arrêter à une expression avec une **double somme**. La réduire à la somme simple de l'énoncé (comme dans les solutions) est technique et hors exercice.
- **4 c** : partir des formules données plus haut dans l'énoncé.

Rappel : le filtre de Wiener

Key formulas

Entrée $X(n)$ WSS, sortie $Y(n) = \sum_k h(k)X(n-k)$, réponse désirée $D(n)$, erreur $e(n) = D(n) - Y(n)$, on minimise $\varepsilon = E[e^2(n)]$:

$$\sum_l h(l)R_X(k-l) = R_{DX}(k) \quad \Rightarrow \quad H(f) = \frac{S_{DX}(f)}{S_X(f)}$$

(non causal), ou, en causal, les **équations de Wiener-Hopf** : $\sum_{l\geq 0} h(l)R_X(k-l) = R_{DX}(k)$ pour $k \geq 0$.

Key idea

Exercice 8 (filtre de prédiction ci-dessous) : l'erreur $e(n)$ et l'entrée X sont **orthogonales**. C'est ce que disent les équations : pour chaque k où le filtre a un coefficient, $E[e(n)X(n-k)] = R_{DX}(k) - \sum_l h(l)R_X(k-l) = 0$.

Expliqué simplement

Le meilleur filtre est celui dont l'erreur restante n'a **plus rien en commun** avec ce qu'on a vu à l'entrée : s'il restait un lien, on pourrait s'en servir pour corriger encore un peu l'erreur, et le filtre ne serait pas le meilleur.

Application : restauration d'image

Definition

Modèle de dégradation (« image » à une dimension pour commencer) :

$$X(n) = \sum_i b(i)S(n-i) + N(n)$$

S : image idéale, X : image dégradée, b : filtre de dégradation (mouvement de la caméra…) de transmittance $B(f)$, N : bruit additif. On cherche $H(f)$ tel que la sortie Y soit proche de S.

Warning

Filtrage inverse $H_{\text{inv}}(f) = 1/B(f)$: $\tilde{Y}(f) = \tilde{S}(f) + \tilde{N}(f)/B(f)$ (transformées de Fourier des signaux, pas des densités spectrales). Si $B(f)$ est très petit à certaines fréquences, le bruit $\tilde{N}/B$ explose et écrase l'image. Et si l'image est seulement bruitée ($B = 1$), il ne fait rien.

Key formulas

Wiener, avec $D = S$, N centré et indépendant de S :

$$S_{DX}(f) = B^\star(f)S_S(f) \qquad S_X(f) = |B(f)|^2 S_S(f) + S_N(f)$$

$$H(f) = \frac{1}{B(f)} \cdot \frac{|B(f)|^2 S_S(f)}{|B(f)|^2 S_S(f) + S_N(f)}$$

Une cascade : un filtre qui **réduit le bruit**, puis le **filtre inverse**. Sans bruit, c'est le filtre inverse ; avec $B = 1$, c'est le Wiener qui débruite. En 2D, même formule en (f_1, f_2).

Key idea

Limites : le critère des moindres carrés n'est pas celui de l'œil (variantes avec des paramètres α, β), une image n'est pas stationnaire (version adaptative, algorithme LMS), et il faut connaître S_S et S_N.

Expliqué simplement

Une photo floue et granuleuse. Le filtre inverse « défloute » à fond, mais il amplifie aussi le grain là où le flou avait presque tout effacé : catastrophe. Wiener est plus malin : il ne défloute que là où le signal est plus fort que le bruit, et laisse tomber les fréquences où il n'y a plus que du grain.

Application : codage PCM et DPCM

Definition

PCM (MIC) : on quantifie chaque échantillon directement, en l'arrondissant au $i\Delta$ le plus proche (Δ : pas de quantification). Simple et rapide, mais n'exploite pas la redondance : environ 64 kbps pour la parole.

Bruit de quantification $q(n) = X(n) - Q(X(n))$ (en négligeant les dépassements de plage) : sous des hypothèses raisonnables, uniforme sur $[-\Delta/2, \Delta/2]$, indépendant d'un échantillon à l'autre, donc un **bruit blanc** :

$$\sigma_q^2 = \frac{\Delta^2}{12} \qquad R_q(k) = \sigma_q^2 \delta(k) \qquad \mathrm{SNR} = \frac{\sigma_X^2}{\sigma_q^2}$$

Definition

DPCM : on **prédit** $X(n)$ à partir des échantillons précédents, $Y(n)$, et on ne quantifie et transmet que l'erreur de prédiction $e(n) = X(n) - Y(n)$. Le récepteur refait la même prédiction et ajoute l'erreur reçue.

Le prédicteur minimise la variance de e, donc $\sigma_e^2 \ll \sigma_X^2$: pour le même nombre de bits, bruit de quantification bien plus faible ; pour la même qualité, moins de bits (environ 16 kbps).

Key idea

En pratique, on prédit à partir des échantillons **quantifiés** $X_q(n-1), \dots$ (le prédicteur est dans la boucle autour du quantificateur) : les erreurs de quantification sont prises en compte et ne s'accumulent pas.

Expliqué simplement

Au lieu d'envoyer chaque valeur en entier, on envoie seulement « **la surprise** » : la différence entre la vraie valeur et ce qu'on pouvait deviner. Comme la parole change doucement, la surprise est petite, donc il faut moins de bits pour l'écrire.

Filtre de prédiction linéaire

Key formulas

$X(n)$ WSS. On estime $X(n)$ à partir des m échantillons précédents :

$$Y(n) = \sum_{k=1}^{m} h(k) X(n-k)$$

en minimisant $\varepsilon = E[(X(n) - Y(n))^2]$: Wiener avec $D(n) = X(n)$, donc $R_{DX}(k) = R_X(k)$, filtre à réponse impulsionnelle finie. Les équations de Wiener-Hopf deviennent les **équations de Yule-Walker** :

$$\sum_{l=1}^{m} h(l) R_X(k-l) = R_X(k), \quad 1 \le k \le m$$

Avec $\mathbf{r}_X = [R_X(1) \dots R_X(m)]^T$ et $\mathbf{h} = [h(1) \dots h(m)]^T$:

$$\Phi_X \mathbf{h} = \mathbf{r}_X \qquad \mathbf{h} = \Phi_X^{-1} \mathbf{r}_X$$

Key idea

Il faut inverser la matrice de corrélation Φ_X (symétrique, Toeplitz).

- Symétrique : $\Phi_X = Q^T \Lambda Q$ avec Q orthogonale (décomposition de **Karhunen-Loève**, qui décorrèle les échantillons), $\Phi_X^{-1} = \sum_i \lambda_i^{-1} \mathbf{q}_i \mathbf{q}_i^T$. Élégant, mais calculer les valeurs propres coûte cher.
- En pratique : l'algorithme de **Levinson**, qui exploite la structure de Toeplitz.

Expliqué simplement

Deviner la prochaine valeur comme un mélange des m dernières. Les bons poids $h(k)$ se trouvent en résolvant un petit système linéaire, qui ne demande que l'autocorrélation : à quel point une valeur ressemble à celle d'il y a 1, 2, ..., m pas.

Application : vocodeur LPC

Definition

Synthèse de la parole (modèle simplifié) : un filtre AR de coefficients a_k, excité par $u(n)$:

- sons **voisés** (/a/, /i/, /v/) : train périodique d'impulsions d'amplitude σ (les cordes vocales), de période le **pitch** ;
- sons **non voisés** (/f/, /s/, /t/) : bruit blanc de variance σ^2.

Sur des blocs courts (10 à 30 ms), la parole est considérée stationnaire.

Key formulas

Vocodeur (*voice coder*) : pour chaque bloc, l'émetteur ne transmet que : voisé ou non, le pitch, le gain σ, et les coefficients de prédiction $h(k)$ (estimées des $-a_k$, ordre $m = p$). Le récepteur resynthétise la parole avec le modèle.

Débit d'environ 4.8 kbps, voire 2.4 kbps (au lieu de 16), mais la parole devient **synthétique**.

Expliqué simplement

Au lieu d'envoyer le son, on envoie la **recette** pour le refaire : « une voix grave, qui dit un son en /a/, à tel volume ». À l'arrivée, une petite machine reproduit le son à partir de la recette. Très économique, mais ça sonne comme un robot.

Processus de comptage

Definition

$\{N(t), t \in \mathbb{R}^+\}$ est un **processus de comptage** si $N(t)$ est le nombre d'évènements dans $[0, t]$:

1. temps continu, valeurs entières ≥ 0 ;
2. $N(0) = 0$;
3. $t_1 < t_2 \Rightarrow N(t_1) \le N(t_2)$;
4. le nombre d'évènements dans $]t_1, t_2]$ est $N(t_2) - N(t_1)$.

Key formulas

Deux processus associés (temps discret, valeurs continues) :

- $S(n)$: **temps d'arrivée** du n-ième évènement ;
- $T(n)$: **temps entre arrivées**, entre le n-ième et le $(n+1)$-ième.

$$N(t) \geq n \Leftrightarrow S(n) \leq t \qquad S(n) = \sum_{m=0}^{n-1} T(m)$$

$$T(n) = S(n+1) - S(n)\ (n \geq 1), \quad T(0) = S(1)$$

$$N(t) = \max\{n : S(n) \leq t\} \qquad S(n) = \inf\{t : N(t) = n\}$$

Expliqué simplement

Un compteur à l'entrée d'un magasin. $N(t)$: combien de clients sont entrés depuis l'ouverture. $S(n)$: à quelle heure est entré le n-ième. $T(n)$: combien de temps on a attendu entre deux clients. Trois façons de raconter la même histoire.

Processus de Poisson : deux définitions

Definition

Définition 1. Poisson homogène d'intensité $\lambda > 0$: un processus de comptage avec

- **H1. accroissements indépendants** (des intervalles disjoints sont indépendants) ;
- **H2. homogène** (accroissements stationnaires) : $P(N(t+T) - N(t) = n_0) = P(N(T) = n_0)$;
- **H3.** pas d'arrivées groupées :

$$P(N(\Delta t) = 1) = \lambda \Delta t + o(\Delta t)$$

$$P(N(\Delta t) \geq 2) = o(\Delta t)$$

donc $P(N(\Delta t) = 0) = 1 - \lambda \Delta t + o(\Delta t)$ (f est $o(\Delta t)$ si $f(\Delta t)/\Delta t \to 0$).

Definition

Définition 2. H1, et **H2'** : pour tous $t, T \geq 0$,

$$P(N(t+T) - N(t) = n) = \frac{(\lambda T)^n}{n!} e^{-\lambda T}$$

Théorème 1 : les deux définitions sont équivalentes.

Expliqué simplement

Des clients qui arrivent **complètement au hasard** : ce qui se passe dans un intervalle n'influence pas un autre (H1), toutes les heures se valent (H2), et deux clients n'arrivent jamais pile en même temps (H3). Ces trois règles de bon sens suffisent à imposer que le nombre de clients dans une durée T suit une loi de Poisson de moyenne λT.

Équations de Kolmogorov

Key formulas

$p_n(t) = P(N(t) = n)$. Avec H1–H3 :

$$p_0(t+\Delta t) = P(N(\Delta t) = 0)\, p_0(t) = (1 - \lambda \Delta t + o(\Delta t))\, p_0(t)$$

et, par les probabilités totales, pour $n > 0$:

$$p_n(t + \Delta t) = (1 - \lambda \Delta t) p_n(t) + \lambda \Delta t\, p_{n-1}(t) + o(\Delta t)$$

En divisant par $\Delta t \to 0$: les **équations de Kolmogorov**

$$\frac{dp_0}{dt}(t) = -\lambda p_0(t) \qquad \frac{dp_n}{dt}(t) = -\lambda p_n(t) + \lambda p_{n-1}(t)$$

avec $p_0(0) = 1$, $p_n(0) = 0$ pour $n \geq 1$.

Key formulas

Solution (par récurrence sur n, ou par la fonction génératrice, exercice 1) :

$$p_0(t) = e^{-\lambda t} \qquad p_n(t) = \frac{(\lambda t)^n}{n!} e^{-\lambda t}$$

$N(t)$ suit la loi de Poisson du module 1 (la limite de la binomiale) ; avec H2, on obtient H2', d'où l'équivalence des définitions.

Expliqué simplement

On regarde ce qui peut se passer pendant un tout petit instant Δt : soit rien, soit exactement un client. Pour avoir n clients à $t + \Delta t$, soit on en avait déjà n et personne n'arrive, soit on en avait $n-1$ et un arrive. Ça donne des équations qui disent comment les probabilités **coulent** de case en case avec le temps.

Propriétés du processus de Poisson

Key formulas

- **P1.** $G_N(z;t) = e^{\lambda t(z-1)}$, $\mu_N(t) = \lambda t$, $\sigma_N^2(t) = \lambda t$.
- **P2.** $R_N(t_1, t_2) = \lambda^2 t_1 t_2 + \lambda \min(t_1, t_2)$ (exercice 2 : écrire $N(t_2) = N(t_1) + (N(t_2) - N(t_1))$ pour $t_1 \leq t_2$).
- **P3. Superposition** : M processus de Poisson indépendants de taux λ_i donnent un processus de Poisson de taux $\lambda = \sum_i \lambda_i$.
- **P4. Décomposition** : si chaque évènement va au sous-processus i avec probabilité p_i ($\sum p_i = 1$), on obtient M processus de Poisson **indépendants** de taux $p_i \lambda$.

Key idea

Preuve de P4 ($M = 2$). Sachant $N(t) = n_1 + n_2$, $N_1(t)$ est binomiale de paramètre p_1 :

$$P(N_1 = n_1, N_2 = n_2) = \binom{n_1+n_2}{n_1} p_1^{n_1} p_2^{n_2} \frac{(\lambda t)^{n_1+n_2}}{(n_1+n_2)!} e^{-\lambda t}$$
$$= \frac{(p_1\lambda t)^{n_1}}{n_1!} e^{-p_1\lambda t} \cdot \frac{(p_2\lambda t)^{n_2}}{n_2!} e^{-p_2\lambda t}$$

Le produit montre l'indépendance; en sommant sur n_2, N_1 est Poisson de taux $p_1\lambda$.

Expliqué simplement

Superposition : deux portes, chacune avec des clients au hasard; tous ensemble, c'est encore des clients au hasard, avec les taux qui s'additionnent. **Décomposition** : on tire à pile ou face pour envoyer chaque client à gauche ou à droite; chaque côté reçoit des clients au hasard, et les deux côtés ne s'influencent même pas.

Temps d'arrivée et temps entre arrivées

Key formulas

P5. Les temps d'arrivée $S(n)$ suivent une loi d'**Erlang** (exercice 4) :

$$f_{S(n)}(s) = \frac{\lambda(\lambda s)^{n-1}}{(n-1)!} e^{-\lambda s}, \quad s \geq 0$$

P6. Les temps entre arrivées $T(n)$ sont des **exponentielles i.i.d.** de moyenne $1/\lambda$:

$$f_{T(n)}(t) = \lambda e^{-\lambda t}, \quad t \geq 0$$

Key idea

Preuve de P6. $P(T(0) > t) = P(N(t) = 0) = e^{-\lambda t}$. Pour $n \geq 1$, par les accroissements indépendants, $T(n)$ est indépendant du passé :

$$P(T(n) > t) = P(N(s_n + t) - N(s_n) = 0 | S(n) = s_n)$$
$$= P(N(t) = 0) = e^{-\lambda t}$$

Définition 3. Avec des $T(n)$ exponentielles i.i.d. de moyenne $1/\lambda$, $S(n) = \sum_{m=0}^{n-1} T(m)$ et $N(t) = \max\{n : S(n) \leq t\}$: processus de Poisson d'intensité λ. L'exponentielle est la seule v.a. continue sans mémoire.

Expliqué simplement

Le temps entre deux clients est **exponentiel** : c'est l'attente du bus qui n'a pas de mémoire. Et l'heure d'arrivée du n-ième client, c'est la somme de n attentes exponentielles : une Erlang (la Gamma du module 1).

Poisson sur $\mathbb{R}$ et bruit impulsif

Definition

Poisson sur $\mathbb{R}$ (processus ponctuel, pour les dispositifs physiques ou optiques) : $N(t)$ défini pour tout t, avec $N(0) = 0$ (donc $N(t) \leq 0$ pour $t \leq 0$), $\cdots \leq S(-1) \leq S(0) \leq 0 < S(1) \leq \ldots$, $T(n) = S(n+1) - S(n)$ pour $n \in \mathbb{Z}$. Les $T(n)$ sont encore exponentielles i.i.d. (y compris $T(0) = S(1) - S(0)$, par l'absence de mémoire), et P1 à P4 restent vraies.

Bruit impulsif de Poisson (*shot noise*), h déterministe :

$$X_h(t) = \sum_{n=-\infty}^{\infty} h(t - S(n))$$

Composé : $\hat{X}_h(t) = \sum_n A_n h(t - S(n))$, A_n i.i.d. indépendants de N. Avec h l'échelon unité, $X_h = N$.

Key formulas

Lemme 1 (dérivée d'un processus, même non stationnaire) : si $X'(t) = \frac{dX}{dt}(t)$,

$$\mu_{X'}(t) = \frac{d\mu_X}{dt}(t) \qquad R_{X'}(t_1, t_2) = \frac{\partial^2 R_X}{\partial t_1 \partial t_2}(t_1, t_2)$$

Avec $h = \delta$, $X_\delta = dN/dt$ (un Dirac à chaque arrivée) :

$$\mu_{X_\delta} = \lambda \qquad R_{X_\delta}(t_1, t_2) = \lambda^2 + \lambda\delta(t_1 - t_2)$$
$$S_{X_\delta}(f) = \lambda^2\delta(f) + \lambda$$

Key formulas

Pour un h quelconque (absolument intégrable), $X_h = h * X_\delta$, donc $S_{X_h}(f) = \lambda^2 H^2(0)\delta(f) + \lambda|H(f)|^2$. **Théorème de Campbell** :

- **P7.** $\mu_{X_h} = \lambda H(0) = \lambda \int h(s)\,ds$, $\sigma^2_{X_h} = \lambda \int |H(f)|^2 df = \lambda \int h^2(s)\,ds$ (Parseval).
- **P8.** Composé : $\mu = \lambda\mu_A \int h(s)\,ds$, $\sigma^2 = \lambda(\mu_A^2 + \sigma_A^2) \int h^2(s)\,ds$.

Expliqué simplement

Chaque arrivée (un électron, un photon) donne une petite **bosse** h dans le courant, et le courant total est la somme de toutes les bosses. Campbell dit tout simplement : la moyenne, c'est (nombre de bosses par seconde) $\times$ (aire d'une bosse); la variance, c'est (nombre de bosses par seconde) $\times$ (aire de la bosse au carré).

Résumé de la semaine

Summary

Exercices M4 : 1, 4; 5, 8, 9.

À retenir

- Wiener : $H = S_{DX}/S_X$; erreur orthogonale à l'entrée ; restauration $H = \frac{1}{B} \cdot \frac{|B|^2 S_S}{|B|^2 S_S + S_N}$.
- PCM : $\sigma_q^2 = \Delta^2/12$; DPCM : on quantifie l'erreur de prédiction.
- Prédiction : Yule-Walker $\Phi_X \mathbf{h} = \mathbf{r}_X$ (Toeplitz, Levinson) ; vocodeur LPC.
- Poisson : H1 + H2 + H3 $\Leftrightarrow$ H1 + H2' ; Kolmogorov ; $p_n(t) = \frac{(\lambda t)^n}{n!} e^{-\lambda t}$.
- $\mu = \sigma^2 = \lambda t$, $R_N = \lambda^2 t_1 t_2 + \lambda \min(t_1, t_2)$; superposition, décomposition.
- $S(n)$ Erlang, $T(n)$ exponentielles i.i.d.
- Shot noise et Campbell : $\mu = \lambda \int h$, $\sigma^2 = \lambda \int h^2$.